Отчёт по лабораторной работе № 13

Семафоры, имитация man и генерация случайных последовательностей

Abstract

В данном отчёте представлены результаты выполнения лабораторной работы по теме: синхронизация процессов с помощью семафоров, имитация команды man и генерация случайных последовательностей букв.

Описаны механизм семафоров с использованием файлов-блокировок, скрипт для работы с командой man, а также генерация случайных строк с использованием встроенной переменной $RANDOM.

Сделаны выводы о достижении поставленной цели и о применении полученных навыков для синхронизации процессов и генерации данных в Linux.

Введение

1. Цель работы
2. Задание и последовательность выполнения работы > 2.1. Скрипт с использованием getopts (поиск с параметрами) > 2.2. Программа на языке C и скрипт для анализа выходного кода > 2.3. Скрипт для создания/удаления нумерованных архивов > 2.4. Скрипт для архивирования с помощью tar и фильтрации по дате
3. Контрольные вопросы
4. Выводы
5. Список литературы

Цель работы

Целью данной лабораторной работы является изучение основ программирования в оболочке ОС UNIX и получение навыков написания более сложных командных файлов с использованием логических управляющих конструкций и циклов.

#Задание и последовательность выполнения работы

В этом разделе описываются скрипты, которые необходимо разработать для данного упражнения, а также шаги и команды, необходимые для их реализации.

2.1. Скрипт с использованием getopts (поиск с параметрами)

Задача: Создайте скрипт, который анализирует командную строку со следующими параметрами и выполняет поиск в файле на основе указанного шаблона. Параметры для реализации:

- -i (входной файл): чтение данных из указанного файла
- -o (выходной файл): запись вывода в указанный файл
- -p (шаблон): поиск по шаблону
- -C: регистрозависимый
- -n: отображение номеров строк

Как работает getopts: getopts — это встроенная команда Bash, которая позволяет структурированно анализировать параметры командной строки. Параметры, которые ожидают аргумент, должны сопровождаться двоеточием (:) в строке параметров. Если параметр имеет аргумент, getopts сохраняет его значение в переменной OPTARG.

Шаг 1: Создание скрипта

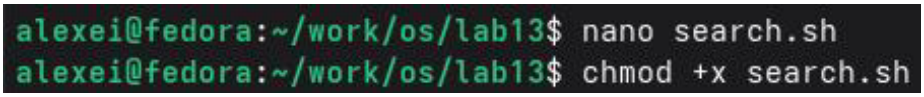
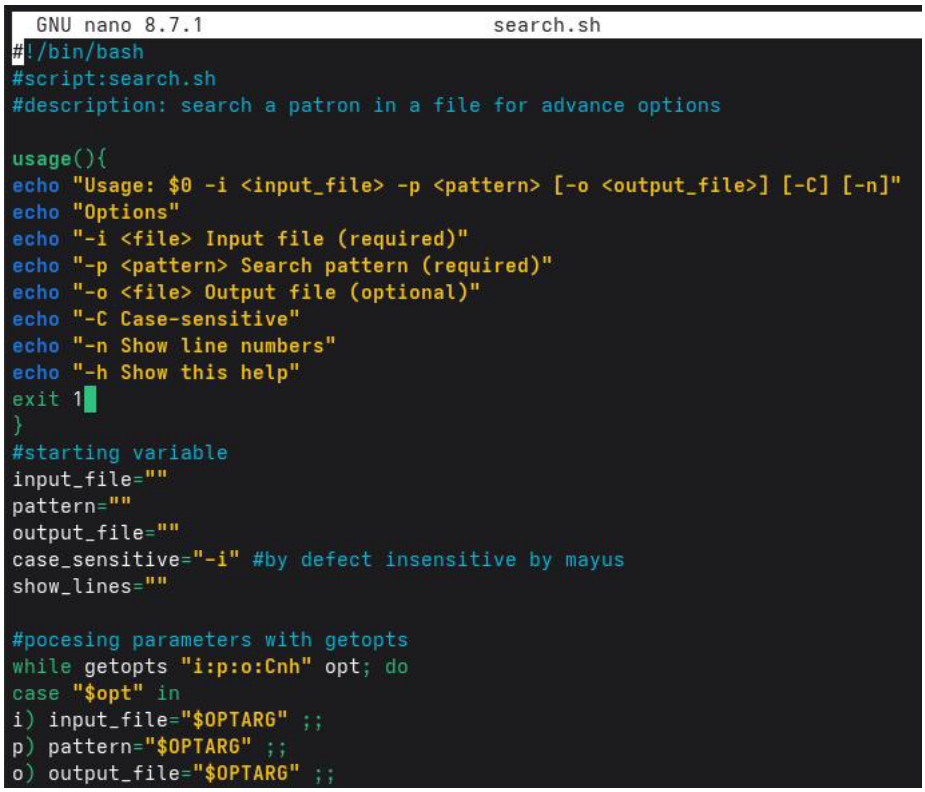


Figure 1: Page 4 image

Код скрипта:



```
  GNU nano 8.7.1                          search.sh
#!/bin/bash
#script:search.sh
#description: search a patron in a file for advance options

usage(){
echo "Usage: $0 -i <input_file> -p <pattern> [-o <output_file>] [-C] [-n]"
echo "Options"
echo "-i <file> Input file (required)"
echo "-p <pattern> Search pattern (required)"
echo "-o <file> Output file (optional)"
echo "-C Case-sensitive"
echo "-n Show line numbers"
echo "-h Show this help"
exit 1
}
#starting variable
input_file=""
pattern=""
output_file=""
case_sensitive="-i" #by defect insensitive by mayus
show_lines=""

#pocesing parameters with getopts
while getopts "i:p:o:Cnh" opt; do
case "$opt" in
i) input_file="$OPTARG" ;;
p) pattern="$OPTARG" ;;
o) output_file="$OPTARG" ;;
```

```
  GNU nano 8.7.1                          search.sh
o) output_file="$OPTARG" ;;
c) case_sensitive="" ;;
n) show_lines="-n" ;;
h) usage ;;
?) echo "error: invalid options -$optarg" >&2; usage ;;
:) echo "error: the option -$OPTARG required an argument" >&2; usage ;;
esac
done
#verificated obligatory arguments
if [-z "$input_file"] || [-z "$pattern"]; then
echo "error: Required -i (incoming file) and -p (pattern)." >&2
usage
fi
#verificated that the incoming file exist
if [! -f "$input_file"]; then
echo "error: the file '$input_file' dont exist." >&2
exit 1
fi

#Performing a search using grep
if [-n "$output_file"; then
grep $case_sensitive $show_lines "$pattern" "$input_file" > "$output_file"
echo "The results are saved in '$output_file'"
else
grep $case_sensitive $show_lines "$pattern" "input_file"
fi
```

Шаг 2: Запуск скрипта

```
alexei@fedora:~/work/os/lab13$ ./search.sh -i file.txt -p "error"
./search.sh: línea 37: [-z: orden no encontrada
./search.sh: línea 37: [-z: orden no encontrada
./search.sh: línea 42: [!: orden no encontrada
./search.sh: línea 48: [-n: orden no encontrada
grep: input_file: No existe el fichero o el directorio
alexei@fedora:~/work/os/lab13$ ./search.sh -i file.txt -p "error" -o results.txt
./search.sh: línea 37: [-z: orden no encontrada
./search.sh: línea 37: [-z: orden no encontrada
./search.sh: línea 42: [!: orden no encontrada
./search.sh: línea 48: [-n: orden no encontrada
grep: input_file: No existe el fichero o el directorio
alexei@fedora:~/work/os/lab13$ ./search.sh -i file.txt -p "ERROR" -C -n
error: invalid options -
Usage: ./search.sh -i <input_file> -p <pattern> [-o <output_file>] [-C] [-n]
Options
-i <file> Input file (required)
-p <pattern> Search pattern (required)
-o <file> Output file (optional)
-C Case-sensitive
-n Show line numbers
-h Show this help
```

2.2. Программа на языке C и скрипт для анализа выходного кода

Задача: Создайте программу на языке C, которая определяет, является ли число положительным, отрицательным или равным нулю, и возвращает определенный код завершения. Затем скрипт Bash должен вызвать эту программу и вывести сообщение в зависимости от кода завершения.

Код завершения: В Bash код завершения команды хранится в переменной $?. По соглашению, код 0 означает успех, а код, отличный от 0, — ошибку.

Шаг 1: Создайте программу на языке C

```
alexei@fedora:~/work/os/lab13$ nano check_number.c
```

Figure 2: Page 6 image

Код программы:

```
  GNU nano 8.7.1                                check_number.c
#include <stdio.h>
#include <stdlib.h>

int main(){
int num;
printf("Enter a number:");
scanf("%d", &num);
if (num > 0){
printf("The number %d is positive.\n", num);

exit(0); // Exit code: 0 (sucess)
}else if (num < 0){
printf("The number %d is negative.\n", num);
exit(1); // Exit code: 1
} else {
printf("The number is zero.\n");
exit(2); //Exit code:2 (zero)
}
}
```

Figure 3: Page 6 image

Шаг 2: Скомпилируйте программу

```
alexei@fedora:~/work/os/lab13$ gcc -o check_number check_number.c
```

Figure 4: Page 7 image

Шаг 3: Создайте скрипт, который анализирует код завершения

```
alexei@fedora:~/work/os/lab13$ nano check_and_report.sh
alexei@fedora:~/work/os/lab13$ chmod +x check_and_report.sh
```

Figure 5: Page 7 image

Код скрипта:

```
  GNU nano 8.7.1                    check_and_report.sh
#!/bin/bash
./check_number
case $? in
0) echo "The number entered is positive." ;;
1) echo "The number entered is negative." ;;
2) echo "The number entered is zero." ;;
*) echo "Unknown exit code:$?" ;;
esac
```

Figure 6: Page 7 image

Шаг 4: Запустите скрипт

2.3. Скрипт для создания/удаления нумерованных файлов

Задача: Создать скрипт, который генерирует N последовательно пронумерованных файлов (например, 1.tmp, 2.tmp, …, N.tmp) и может также удалять их.

Шаг 1: Создание скрипта

Код скрипта:

```
alexei@fedora:~/work/os/lab13$ ./check_and_report.sh
Enter a number:8
The number 8 is positive.
The number entered is positive.
```

Figure 7: Page 7 image

```
alexei@fedora:~/work/os/lab13$ nano manage_files.sh
alexei@fedora:~/work/os/lab13$ chmod +x manage_files.sh
```

Figure 8: Page 7 image

```
 GNU nano 8.7.1                          manage_files.sh
 1 #!/bin/bash
 2 #script:manage_files.sh
 3 #description: create o eliminated enumerate files
 4 usage(){
 5 echo "Usage: $0 -c <N> #Create N files"
 6 echo " $0 -d #Delate all created .tmp files"
 7 exit 1
 8 }
 9 #Checking arguments
10 if [$# -eq 0]; then
11 usage
12 fi
13 #Parameter processing
14 while getopts "c:d" opt; do
15 case "$opt" in
16
17 c)
18
19 N="$OPTARG"
20 echo "Creating $N files"
21 for i in $(seq 1 "$N"); do
22 #Create  a file with the formate 01.tmp, 02.tmp and etc.
23 printf -v num "%02d" "$i"
24 touch "${num}.tmp"
25 echo "Files ${num}.tmp has been created"
26 do
27 echo "$N files have been created"
```

```
27 echo "$N files have been created"
28
29 d)
30
31 echo"Deleting .tmp files"
32 rm -f [0-9][0-9].tmp
33 echo "Files have been deleted"
34 *)
35 usage
36 esac
37 done
38
```

Шаг 2: Запуск скрипта

```
alexei@fedora:~/work/os/lab13$ ./manage_files.sh -c 10
Creating 10 files ...
Files 01.tmp has been created
Files 02.tmp has been created
Files 03.tmp has been created
Files 04.tmp has been created
Files 05.tmp has been created
Files 06.tmp has been created
Files 07.tmp has been created
Files 08.tmp has been created
Files 09.tmp has been created
Files 10.tmp has been created
10 files have been created
alexei@fedora:~/work/os/lab13$ ./manage_files.sh -d
Deleting .tmp files
Files have been deleted
```

2.4. Скрипт для архивирования с помощью tar и фильтрации по дате

Задача: Создать скрипт, который архивирует все файлы в каталоге с помощью tar, но только те, которые были изменены за последнюю неделю.

Шаг 1: Создание скрипта

```
alexei@fedora:~/work/os/lab13$ nano backup_recent.sh
alexei@fedora:~/work/os/lab13$ chmod +x backup_recent.sh
```

Figure 9: Page 9 image

Код скрипта:

```
  GNU nano 8.7.1                    backup_recent.sh
#!/bin/bash
#script: backup_recent.sh
#description: archives files ,odified over the past week using tar and find

#cheking the catalog's identify as an arguments
if [ $# -ne 1]; then
echo "Usage $0 <directory>"
echo "Example: $0 /home/user/documents"
exit 1
fi
TARGET_DIR="$1"
ARCHIVE_NAME="backup_$(date +%Y%m%D).tar.gz"
#Checking the existence of a directory
if [! -d "$TARGET_DIR" ]; then
echo "Error: The '$TARGET_DIR' directory does not exits"
exit 1
fi
echo "Files to include (changed in the last 7 days:"
find "$TARGET_DIR" -mtime -7 -type f -printf "%f\n"
#Compress only files modified in the last week
find "$TARGET_DIR" -mtime -7 -type f -exec tar -rvf "$ARCHIVE_NAME" {}\;
echo "File created: $ARCHIVE_NAME"
```

Figure 10: Page 9 image

Шаг 2: Запустите скрипт

Контрольные вопросы

В этом разделе представлены ответы на контрольные вопросы по продвинутому программированию в Bash, которые позволят вам проверить понимание концепций getopts, метасимволов, управляющих структур и циклов.

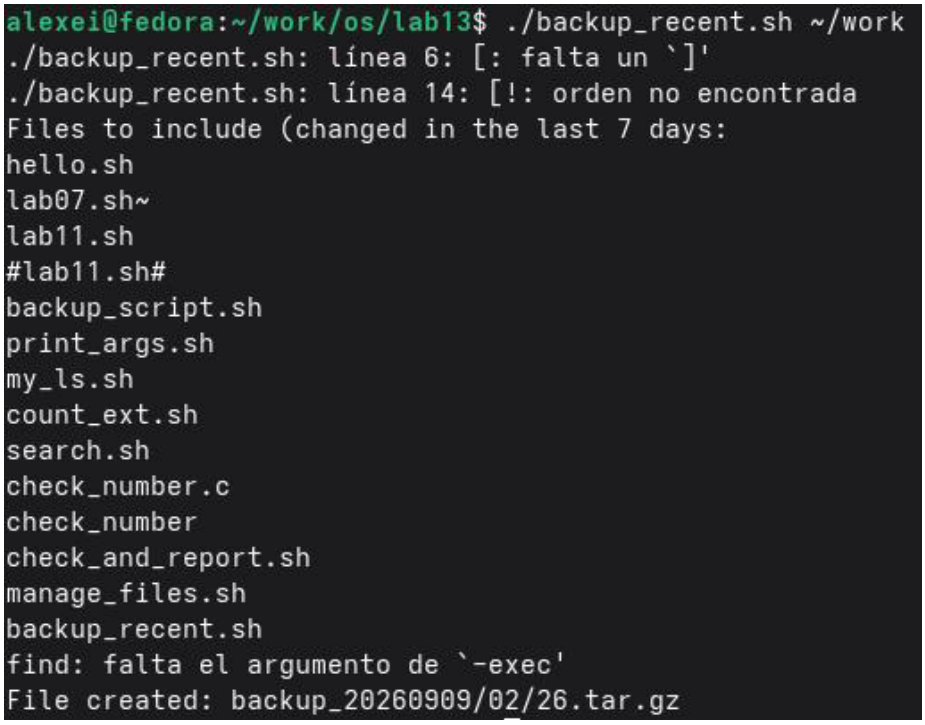


Figure 11: Page 10 image

1. Что такое команда getopts?

Команда getopts — это встроенный инструмент Bash, позволяющий структурированным и стандартным способом анализировать и обрабатывать параметры командной строки в скриптах. Её цель — упростить реализацию пользовательских интерфейсов в скриптах, позволяя скрипту получать аргументы с флагами (например,-i,-o,-p и т.д.) и их соответствующие значения. Основные возможности:

- Обрабатывает однобуквенные параметры с дефисом (-).
- Допускает параметры с аргументами (используя двоеточие в строке параметров).
- Сохраняет значение аргумента в переменной OPTARG.
- Сохраняет индекс следующего аргумента в переменной OPTIND.
- Поддерживает стандартные параметры и обработку ошибок.

2. Какое отношение метасимволы имеют к генерации имён файлов?

Метасимволы (также называемые «подстановочными знаками») в Bash позволяют генерировать имена файлов с использованием шаблонов (глоббинг). Это особенно полезно,когда необходимо ссылаться на несколько файлов одновременно.

Метасимвол	Описание	Пример
*	Соответствует любой строке (включая пустые строки)	*.txt → все файлы .txt
?	Соответствует одному символу	file?.txt → file1.txt, file2.txt → file0.txt,file1.txt
[]	Соответствует набору символов	file[0-9].txt
[^]	Исключает набор символов	file[^0-9].txt → исключает файлы с числовыми значениями

‘Связь с генерацией имен файлов:

- Метасимволы позволяют разворачивать шаблоны в списках файлов.
- Это упрощает операции с несколькими файлами (например, `cp *.pdf ~/documents/`).
- Оболочка выполняет расширение перед выполнением команды.

3. Какие операторы вы знаете?

В Bash есть несколько операторов и управляющих структур:

Структура	Описание	Пример
if	Простое условное выражение	если [условие]; тогда … fi
if-else	Условное выражение с альтернативой	если [условие]; тогда … иначе … fi
if-elif-else	Несколько условий	если [c1]; тогда … elif [c2]; тогда … иначе … fi
case	Множественный выбор	case $var in pattern1) … ;; pattern2) … ;; esac
for	Цикл по списку	for i in list; do … done
while	Условный цикл	while [условие]; do … done
until	Цикл до условия	until [условие]; do … done
break	Выход из цикла	break
continue	Переход к следующей итерации	continue

4. Какие операторы используются для создания цикла?

В Bash для прерывания циклов используются следующие операторы:

- break — Завершает выполнение текущего цикла (полностью выходит из него).

```
for i in {1..10}; do
if [ $i -eq 5 ]; then
break #              i=5
fi
echo $i
done
```

- continue — Переход к следующей итерации цикла (не завершает весь цикл).

```
for i in {1..5}; do
if [ $i -eq 3 ]; then
continue #              i=3
fi
echo $i
done
```

5. Для чего нужны команды `false` и `true`?

Команды `true` и `false` — это встроенные утилиты Bash, возвращающие определенные коды завершения:

- `true` — Всегда возвращает код завершения 0 (успех/истина).
- `false` — Всегда возвращает код завершения 1 (неудача/ложь).

Распространенные варианты использования:

1. Бесконечные циклы:

```
while true; do
echo "Running...
sleep 1
done
```

2. Условные скрипты:

```
if true; then
echo "This always runs
```

3. Отладка и тестирование:

```
command && echo "Success" || echo "Failure"
```

6. Что означает `if test -f mans/s/i.\$s`?

Эта строка проверяет, существует ли файл с динамически генерируемым именем. Давайте проанализируем каждую часть:

Часть	Значение
`if test -f`	Проверяет, существует ли файл и является ли он обычным (-f).
`man`	Фиксированный префикс имени файла.
`$s`	Переменная, содержащая значение расширения или суффикса.
`\\$i`	Экранирует знак доллара (), , `i . | |.| . | |\s`|(), чтобы отображать $s буквально.

Интерпретация:

Эта строка проверяет, существует ли файл с именем man$i. в текущем каталоге.

7. Объясните различные конструкции while и until.

Характеристика	Условие	until
while	Выполняется,пока условие истинно (код завершения 0).	Выполняется до тех пор, пока условие истинно (код завершения 0).
Эквивалент	while [condition]; do … done	while [! condition]; do … done
Типичное использование	Повторять,пока что-то истинно.	Повторять, пока что-то истинно.

Пример:

```
# while:        ,     counter <= 5
counter=1
while [ $counter -le 5 ]; do
echo "while: $counter"
((counter++))
done
# until:        ,     counter > 5
counter=1
until [ $counter -gt 5 ]; do
echo "until: $counter"
((counter++))
done
```

Основное различие:

- while выполняет блок, пока условие истинно.
- Функция `until` выполняет блок кода до тех пор, пока условие не станет истинным (т.е. пока оно ложно).

#Выводы

В ходе выполнения лабораторной работы я изучил основы программирования в оболочке ОС UNIX и научился писать более сложные командные файлы с использованием логических управляющих конструкций и циклов. Я освоил использование команды getopts для анализа командной строки с ключами, создал программу на языке Си и командный файл для анализа кода завершения, разработал скрипт для создания и удаления пронумерованных файлов, а также реализовал скрипт для архивации файлов с фильтрацией по дате изменения с

помощью команд tar и find. Все поставленные задачи выполнены, полученные навыки являются основой для автоматизации сложных задач в операционной системе Linux.

Список литературы

- Baeldung. (2021). Анализ аргументов командной строки в Bash.
- GNU. (2026). Справочное руководство по Bash: Код завершения.
- Zero To Mastery. (2025). Руководство для начинающих по использованию getopts в Bash.
- LabEx. (2025). Как использовать код завершения в скриптах?
- GeeksforGeeks. (2024). Скриптинг в Bash — цикл while. Получено 15 августа 2026 г. с
- *GeeksforGeeks. (2023). Скриптинг в Bash — цикл until. Получено 15 августа 2026 г. с